Atelier 5 — Gestion de versions avec Git

Corrigé des questions lisibles du sujet fourni

Document : Corrigé rédigé à partir des supports fournis

Note sur le support : le PDF reçu contient plusieurs copies visuellement identiques de la même page d'examen. Le corrigé ci-dessous traite les questions lisibles des parties « Exercice Git » et « Gestion de version avec Git », sans ajouter d'autres questions qui ne figurent pas sur la page.

Partie 3 — Exercice Git

1. Voir l'historique des commits

Utiliser git log pour l'historique détaillé ou git log --oneline pour une ligne par commit.

```
git switch main
git pull origin main
git switch -c feature-xyz
```

3. Fusionner feature-xyz dans main

```
git switch main
git merge feature-xyz
```

La fusion se fait depuis la branche cible, ici main. Si des conflits apparaissent, les résoudre dans les fichiers concernés, préparer les fichiers résolus avec git add, puis terminer la fusion avec un commit si Git le demande.

4. Annuler des modifications locales avant commit

```
git restore nom_du_fichier
```

Cette commande abandonne les modifications non préparées du fichier suivi et rétablit la version du dernier commit. Pour plusieurs fichiers suivis, git restore . agit sur le répertoire courant. Si les changements sont déjà dans le staging, il faut d'abord les retirer du staging avec git restore --staged nom_du_fichier, puis restaurer le fichier. Ces opérations font perdre les modifications concernées : vérifier le diff avant de les lancer.

5. Vérifier l'état courant du dépôt

```
git status
```

Cette commande résume la branche courante, les fichiers modifiés, les fichiers non suivis et les changements préparés pour le prochain commit.

Partie 2 — Gestion de version avec Git

1. Créer le dossier depot_local et l'initialiser

```
mkdir depot_local
cd depot_local
git init
git branch -M main
```

Si Git est configuré pour utiliser un autre nom de branche principale, garder ce nom de façon cohérente.

2. Lier le dépôt local à un dépôt distant

Créer d'abord un dépôt vide sur GitHub, puis remplacer le placeholder par l'URL réelle :

```
git remote add origin https://github.com/VOTRE-UTILISATEUR/VOTRE-DEPOT.git
git remote -v
```

Une URL SSH est également possible si la clé SSH du poste est configurée.

3. Ajouter trois fichiers vides puis créer le premier commit

```
touch produit.html style.css script.js
git status
git add produit.html style.css script.js
git status
git commit -m "Initialisation du projet produits"
```

Sous Windows, si touch n'est pas disponible, créer les fichiers depuis l'explorateur ou utiliser PowerShell : New-Item produit.html, style.css, script.js -ItemType File.

4. Créer une branche pour la gestion des produits

```
git switch -c gestion-produits
```

Le sujet écrit « Gestion produits », mais les noms de branches Git ne peuvent pas contenir d'espace. Le nom gestion-produits est donc une forme valide qui représente cette fonction.

5. Modifier les trois fichiers, commiter, revenir sur main et fusionner

Modifier les fichiers avec le contenu souhaité, puis exécuter :

```
git add produit.html style.css script.js
git commit -m "Mise à jour de la gestion produits"
git switch main
git merge gestion-produits
```

Partie 2 — suite : log, remote, push et retour en arrière

6. Afficher le log compact, vérifier origin et publier le travail

```
git log --oneline
git remote show origin
git push --set-upstream origin main
```

Si la branche distante principale est déjà initialisée par un README ou un autre commit indépendant, il peut être nécessaire de récupérer d'abord cet historique et de le réconcilier avant le push. Pour un dépôt distant vide nouvellement créé, la séquence ci-dessus convient généralement.

7. Revenir au premier commit et supprimer les commits suivants et les modifications locales

Identifier le hash du premier commit, puis réinitialiser la branche locale vers ce commit :

```
git log --oneline --reverse
git reset --hard HASH_DU_PREMIER_COMMIT
git status
git log --oneline
```

Attention — opération destructive : git reset --hard retire de la branche locale les commits postérieurs au hash choisi et abandonne les modifications locales suivies non sauvegardées. Faire une sauvegarde ou une branche de secours avant de l'exécuter. Cette commande ne supprime pas à elle seule l'historique déjà publié sur GitHub ; ne pas utiliser de push forcé sur une branche partagée sans autorisation.

Récapitulatif des commandes essentielles

Besoin	Commande principale
Consulter l'historique	git log / git log --oneline
Créer une branche et basculer dessus	git switch -c nom-branche
Changer de branche	git switch main
Fusionner une branche dans la branche courante	git merge nom-branche
Voir l'état du dépôt	git status
Annuler des modifications non préparées	git restore fichier
Associer un dépôt distant	git remote add origin URL
Afficher les dépôts distants	git remote -v
Publier la branche principale	git push --set-upstream origin main

Conclusion

Le flux type est : créer ou cloner un dépôt, préparer les fichiers avec git add, enregistrer un commit, travailler sur une branche, fusionner vers la branche cible et synchroniser avec le dépôt distant. Les commandes qui réécrivent l'historique ou abandonnent des changements doivent être utilisées avec précaution.